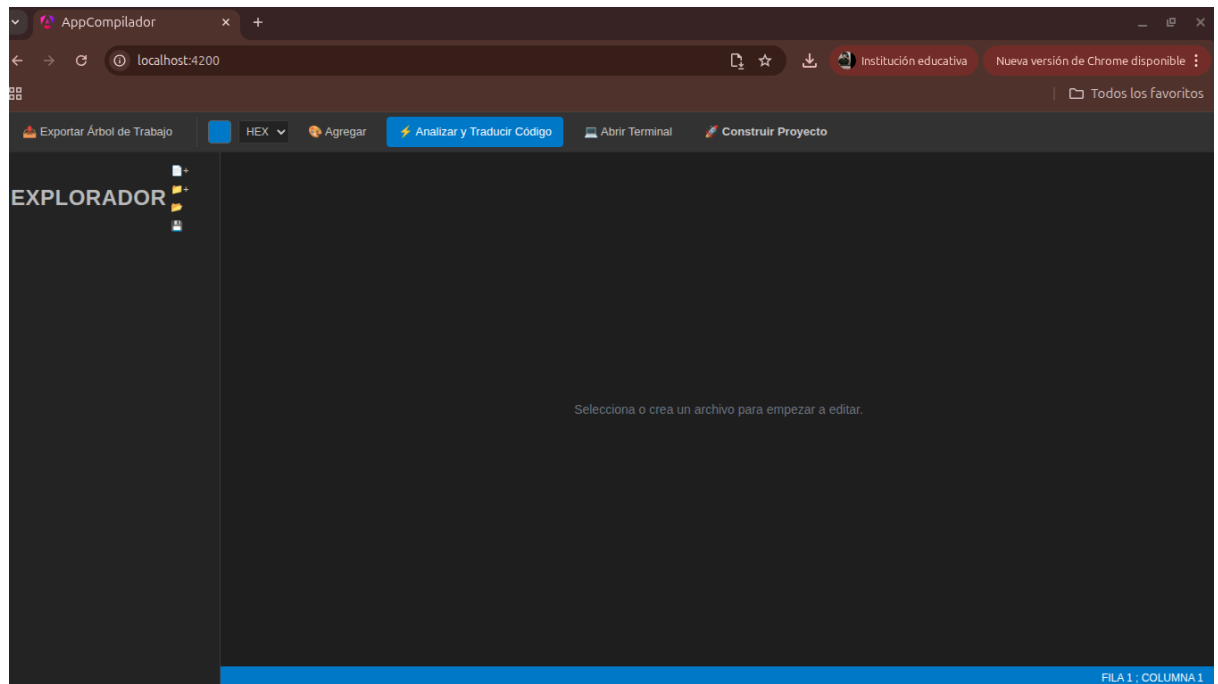


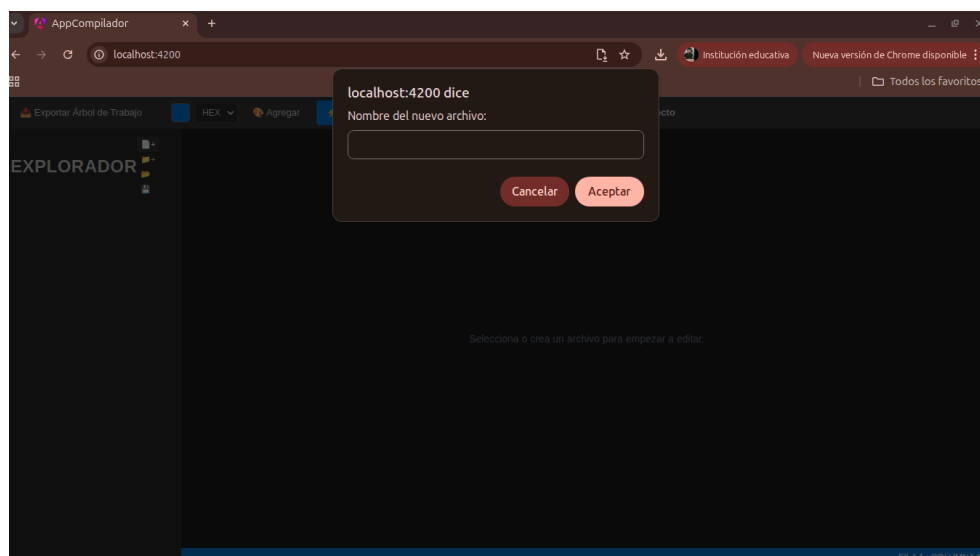
MANUAL DE USUARIO.

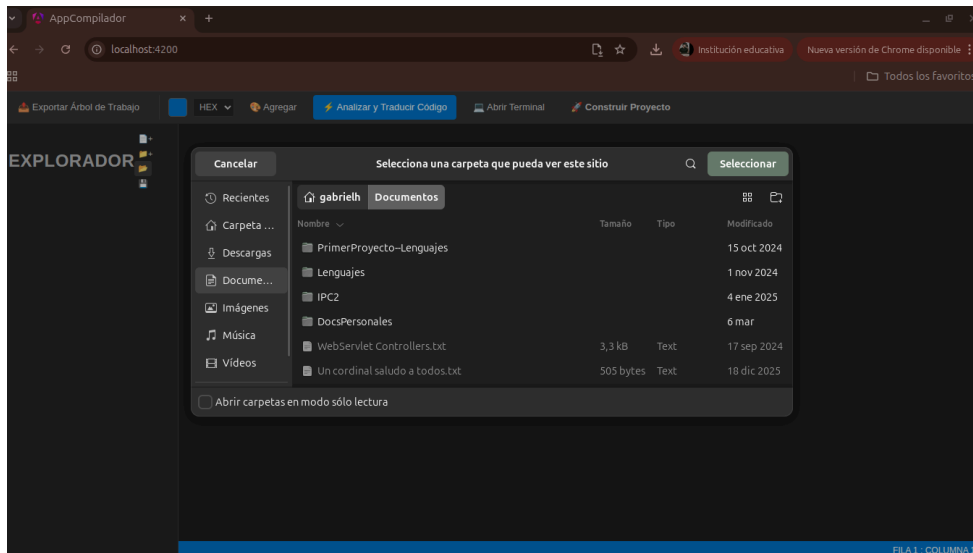
Fredy Jose Gabriel Herrera Funes. Carnet: 202130478

1. Inicializar programa: situarse en la carpeta Frontend el proyecto abrir una terminal y ejecutar el comando “npm run start”, luego situarse en la carpeta Backend del proyecto y ejecutar el comando “node [server.js](#)”
2. Una vez realizado esto aparecerá la siguiente pantalla:



3. Habrás iniciado el YFera IDE, para abrir o crear un archivo-carpeta, se encuentran los botones justo al lado de la palabra EXPLORADOR, al crear un archivo-carpeta se te pedirá un nombre para el archivo-carpeta, y también puedes abrir proyectos ya realizados desde tu computadora:





NOTA: puedes crear más carpetas en las carpetas, y archivos en las carpetas ya existentes, puedes renombrar un archivo o carpeta, mover y eliminar, todo con los botones que aparecen al lado derecho del nombre del archivo.

- Ahora una vez creado tu archivo, debes tomar en cuenta que existen tres lenguajes .y (lenguaje principal), .comp(lenguaje de componentes), .style(lenguaje de estilos), a continuación te doy un ejemplo de cada uno:

LENGUAJE PRINCIPAL:

```
import "./fcb.style";
```

```
import "./vistas.comp";
```

```
/* Variables Globales */
```

```
string equipo = "FC Barcelona";
```

```
/* Arreglos de datos (Aseguramos que todos tengan tamaño 4) */
```

```
string[] nombres = {"Robert Lewandowski", "Pedri", "Ronald Araujo", "Lamine Yamal"};
```

```
string[] posiciones = {"Delantero", "Mediocampista", "Defensa", "Delineante"};
```

```
int[] stats = {18, 5, 45, 6};
```

```
/* Función simulada de base de datos */
```

```
function refrescarDatos() {
    execute `log_action[type="refresh_stats"]`;
    load "./main.y";
}
```

```
/* Ejecución principal */
```

```
main {
```

```

/* 1. Pintamos el encabezado */
@HeaderStats(equipo);

/* 2. Ciclo para dibujar a los 4 jugadores */
for (i = 0 ; i < 4 ; i = i + 1) {
    /* ¡OJO! El orden debe ser exacto al de la vista: Nombre, Posición, Número */
    @PlayerCard(nombres[i], posiciones[i], stats[i]);
}

/* 3. Pintamos el footer pasándole la función */
@FooterStats(refrescarDatos);
}

```

LENGUAJE COMP:

```

/* Componente de Título */
HeaderStats(string tituloEquipo) {
    <fondo-app>[
        T<texto-3>("Plantilla Oficial: $tituloEquipo")
        T<texto-1>("Estadísticas actualizadas de la temporada")
    ]
}

/* Componente de Jugador (El orden de variables es vital: nombre, posicion, goles) */
PlayerCard(string nombre, string posicion, int goles) {
    Switch($posicion) {
        case "Delantero" {
            <tarjeta-delantero>[
                T<texto-2>("★ $nombre")
                T<texto-1>("Posición: $posicion | Goles anotados: $goles")
            ]
        },
        case "Mediocampista" {
            <tarjeta-mediocampista>[
                T<texto-2>("⚽ $nombre")
                T<texto-1>("Posición: $posicion | Asistencias/Goles: $goles")
            ]
        },
        default {
            <tarjeta-defensa>[
                T<texto-2>("🛡️ $nombre")
                T<texto-1>("Posición: $posicion | Balones recuperados: $goles")
            ]
        }
    }
}
}

```

```

}

/* Botón inferior */
FooterStats(function accion) {
  <fondo-app>[
    FORM {
      T<texto-1>("¿Sincronizar nuevos datos desde el servidor?")
    } SUBMIT<btn-actualizar> {
      label: "Cargar Estadísticas"
      function: $accion()
    }
  ]
}

```

LENGUAJE STYLE:

```

/* Contenedor principal con fondo suave */
fondo-app {
  background color = lightgray ;
  padding = 20 ;
  text font = SANS ;
  color
}

/* Tamaños de texto dinámicos: texto-1, texto-2, texto-3 */
@for $i from 1 through 3 {
  texto-$i {
    text size = $i * 12 ;
  }
}

/* Tarjeta base neutral */
tarjeta-base {
  padding = 15 ;
  margin bottom = 10 ;
  border radius = 8 ;
  background color = white ;
}

/* Estilos específicos por posición heredando de la base */
tarjeta-delantero extends tarjeta-base {
  border = 2 solid red ;
  color = red ;
}

```

```

tarjeta-mediocampista extends tarjeta-base {
    border = 2 solid blue ;
    color = blue ;
}

```

```

tarjeta-defensa extends tarjeta-base {
    border = 2 solid black ;
    color = black ;
}

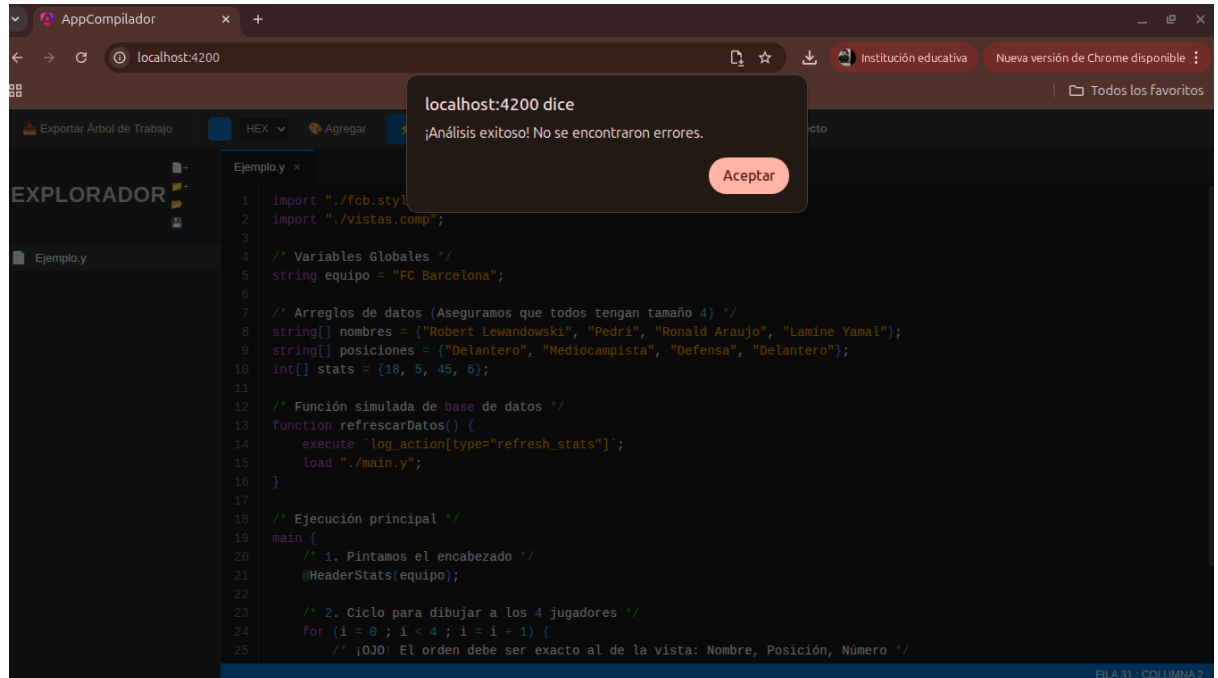
```

```

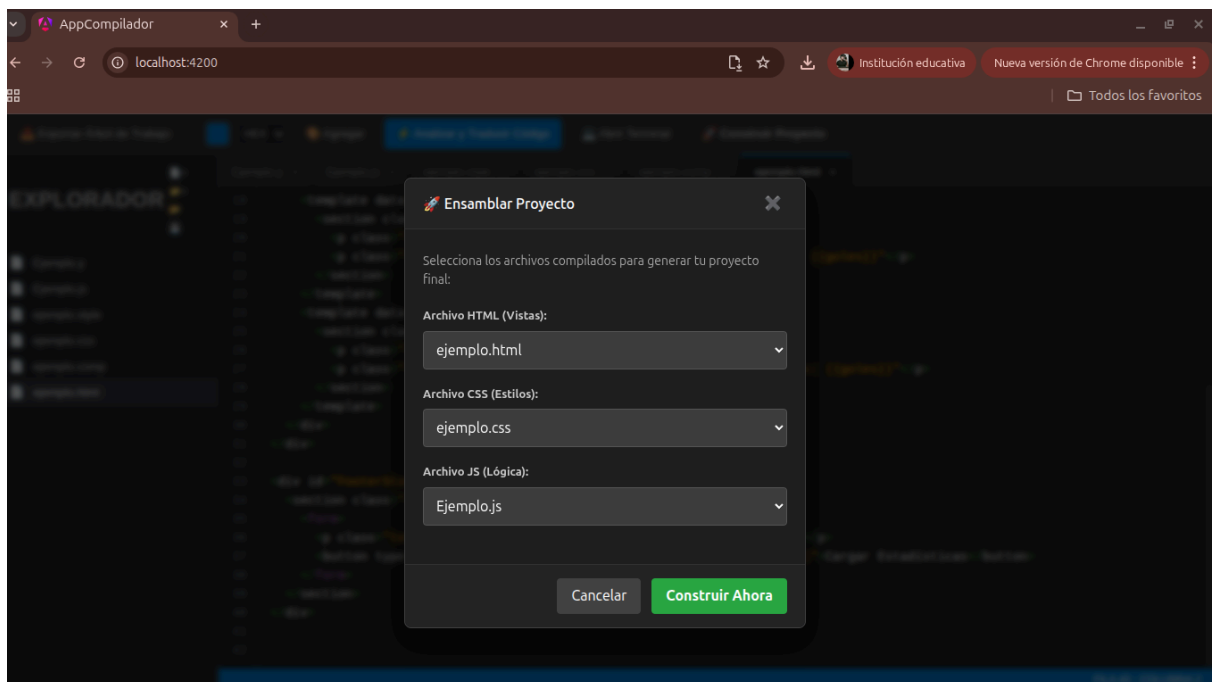
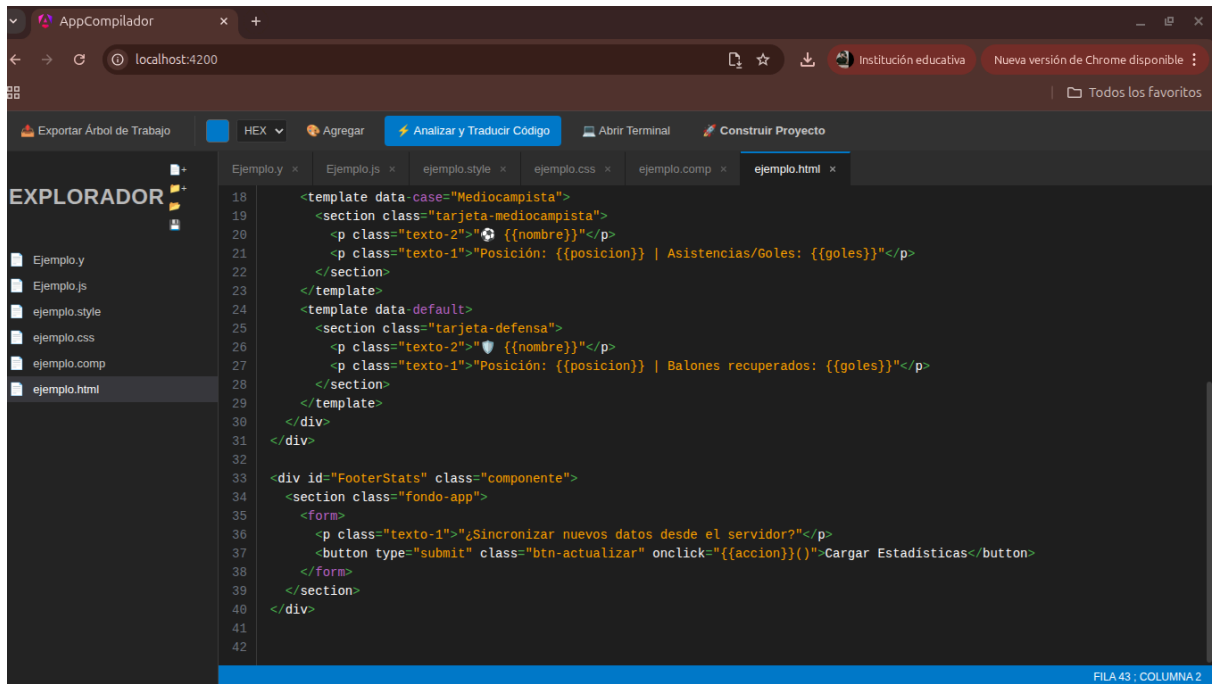
btn-actualizar {
    background color = black ;
    color = white ;
    padding = 12 ;
    border radius = 6 ;
}

```

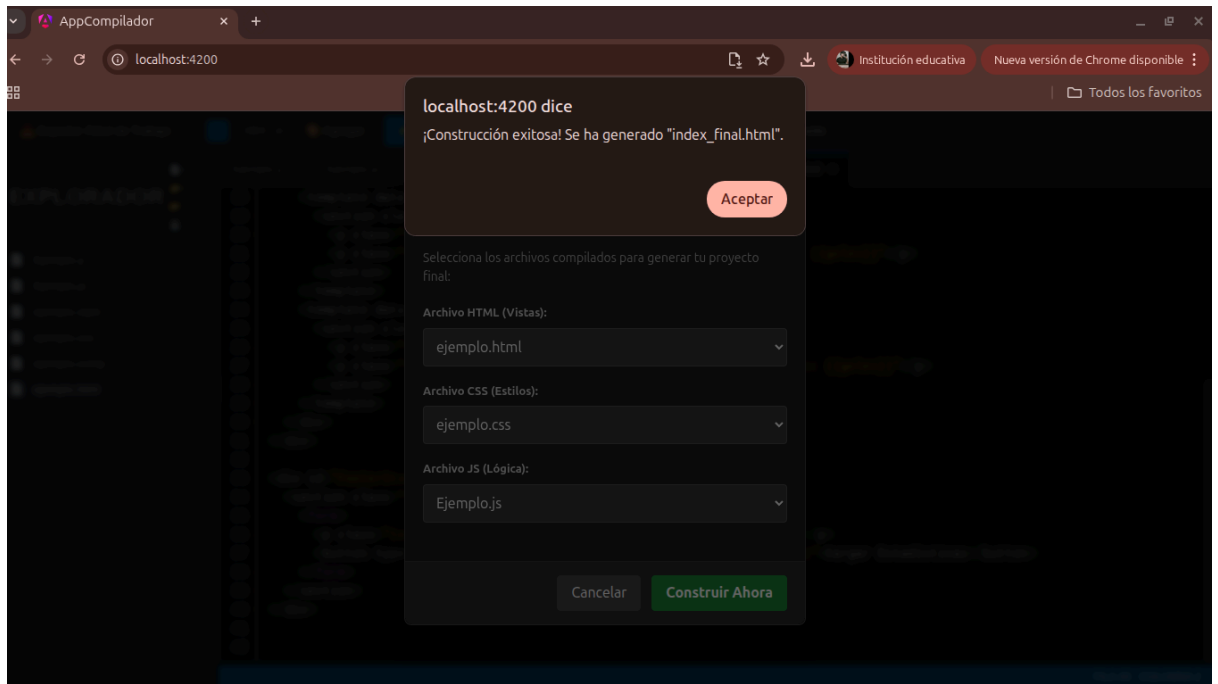
- Una vez ingresado tu código en el archivo, puedes analizarlo y traducirlo, si hay errores en la parte inferior del pantalla se mostrarán, si sale correcto el análisis te mostrará un mensaje de análisis exitoso, y se creará tu archivo .html, .css o .js según el lenguaje que estés traduciendo.



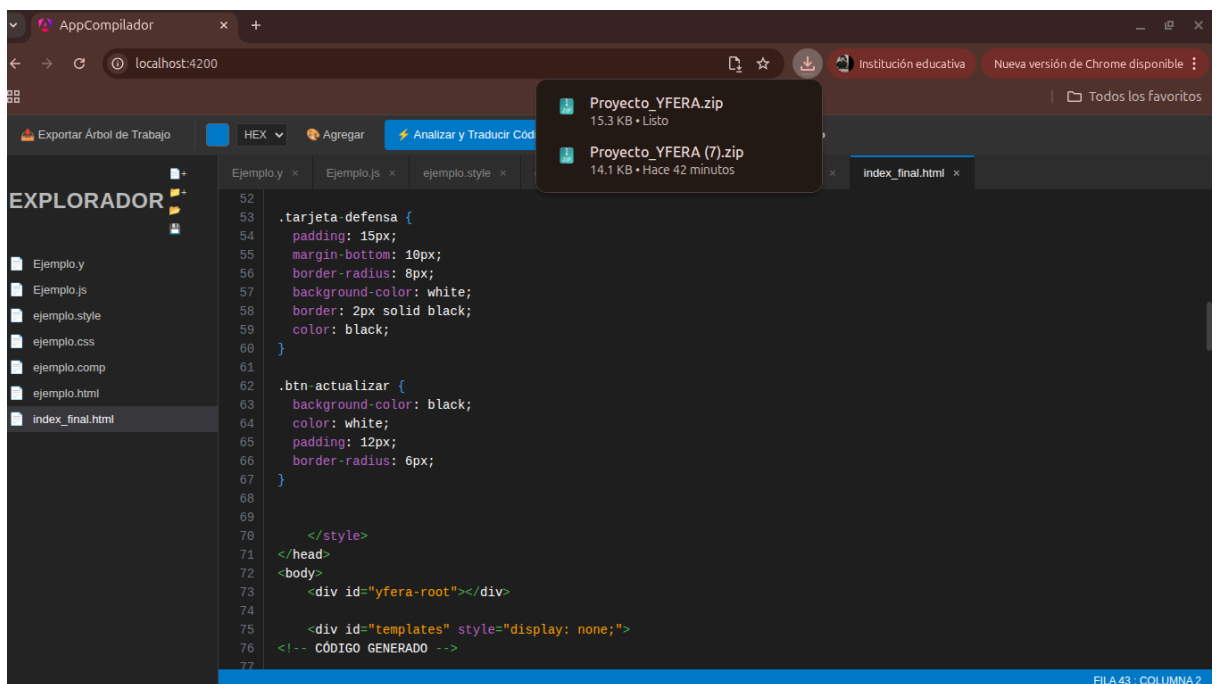
- Una vez teniendo tus archivos .html, .css y .js, puedes construir tu proyecto final, dando click en el botón correspondiente y seleccionando tus archivos:



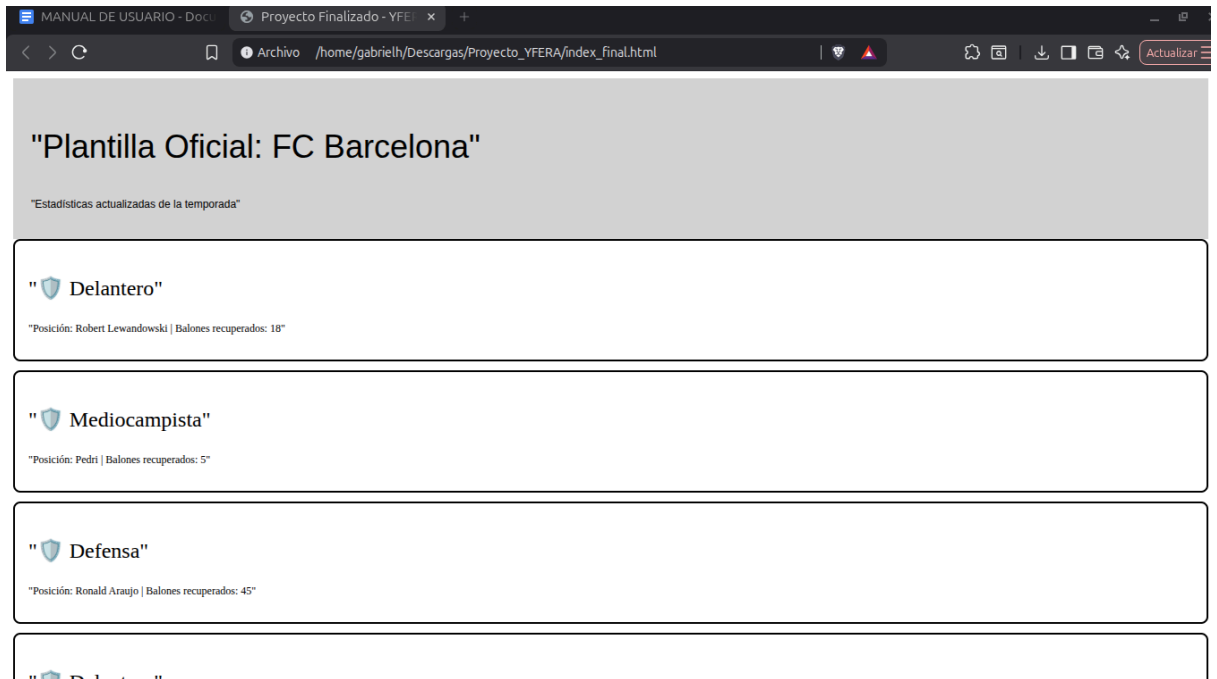
7. Al dar click en construir ahora, se generará un archivo html final, con tu proyecto incluido, listo para su ejecución:



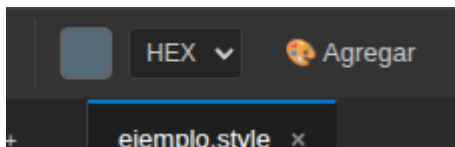
8. También puedes exportar tu árbol de trabajo o todo tu proyecto dando click en el botón correspondiente, al dar click se descargará un archivo .zip, que contendrá tu proyecto:



9. Y así se vería tu html final:

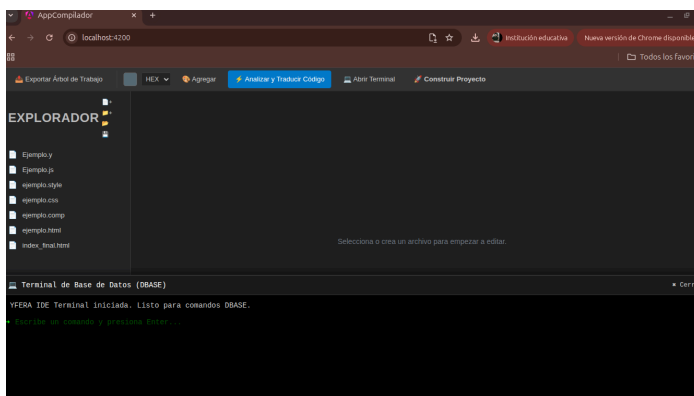


10. También puedes agregar colores desde el selector de colores:

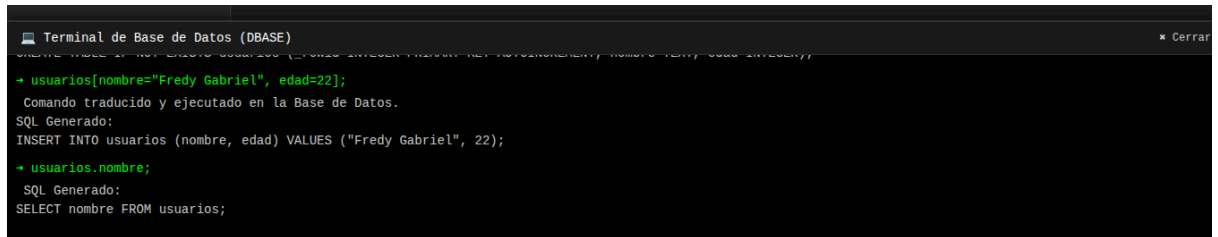


11. Debes seleccionar el color que quieras en el cuadrito, el formato en que lo quieras si HEX(hexadecimal) o RGB, al finalizar tu selección dale click en “Agregar”
12. También puedes interactuar con la base de datos del YFera IDE, abres la terminal, e ingresas tus comandos para la base de datos en el lenguaje indicado, a continuación un ejemplo:

```
TABLE estudiantes COLUMNS carnet=string, creditos=int, promedio=float;
estudiantes[carnet="202012345", creditos=120, promedio=85.5];
estudiantes[carnet="202154321", creditos=90, promedio=78.2];
estudiantes[creditos=125, promedio=86.0] IN 1; estudiantes.carnet;
estudiantes.promedio; estudiantes DELETE 2;
```



13. Al ingresar tus comandos, en la terminal, debes presionar Enter para que se ejecuten, una vez ejecutados se mostrará la acción realizada.



```
Terminal de Base de Datos (DBASE)
CREATE TABLE IF NOT EXISTS usuarios (id INT(11) UNSIGNED PRIMARY KEY AUTOINCREMENT, nombre VARCHAR(255) NOT NULL);
+ usuarios[nombre="Fredy Gabriel", edad=22];
Comando traducido y ejecutado en la Base de Datos.
SQL Generado:
INSERT INTO usuarios (nombre, edad) VALUES ("Fredy Gabriel", 22);
+ usuarios.nombre;
SQL Generado:
SELECT nombre FROM usuarios;
```

14. Y ese sería el funcionamiento del YFera IDE, y su Framework.